ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY

Department of Computer science and Engineering

Mobile Computing and Applications project documentation

Project Title: FindIt - Campus Lost & Found

Section: 3

NAME	ID
1.Naol Shumi	ugr/35082/16
2. Biniyam Teku	ugr/34102/16
3.Menwuyelet Temesgen	ugr/34920/16
4.Nuredin Wario	ugr/35196/16
5.Firomsa Tsegaye	ugr/34458/16

Submitted to: Amir Ibrahim Tahir
Academic Year: 2025/26

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

Lost and found management is a common challenge faced by students and staff in university campuses. Every day, valuable items such as ID cards, keys, phones, chargers, and bags are lost across campus locations including classrooms, libraries, cafeterias, and laboratories. Without a centralized system, recovering lost items relies heavily on informal methods such as posting messages on social media groups like Telegram or asking friends and acquaintances.

These informal methods are inefficient, unreliable, and lack proper documentation. Items often go unclaimed because the owner and the finder have no effective way to connect. As mobile technology becomes increasingly available to students, there is a clear opportunity to develop a dedicated mobile application that streamlines the process of reporting, searching, and recovering lost items on campus.

1.2 Motivation for the Project

The motivation for developing the FindIt application comes from the real-world difficulties students face when they lose items on campus. A survey conducted among 8 students revealed that 100% of respondents had lost something on campus, with keys and ID cards being the most commonly lost items. The majority of students currently rely on Telegram groups to report lost or found items, which is an unstructured and ineffective approach.

Additionally, the survey revealed that when students find someone else's item, 75% post it on Telegram while some simply leave it where it is or keep it. This lack of a proper system creates a significant gap between lost items and their recovery. FindIt aims to bridge this gap through a structured, mobile-first platform.

1.3 Project Overview

FindIt is a mobile application developed using React Native and Node.js/MongoDB that enables university students to report lost items, post found items, search for matching items, and communicate directly with each other to facilitate the return of lost belongings. The application includes an in-app notification system that alerts users when a matching item is posted.

The system supports two main user roles:

Student Users: Can post lost or found items, search and filter items, claim items through a secure verification process, send and receive anonymous messages, edit and delete their own posts, and track item status.

Administrator Users: Can monitor posts, manage users, resolve disputed claims, and view analytics from the same app via role-based routing

Student Users: Can post lost or found items, search and filter items, claim items through a secure verification process, send and receive anonymous messages, edit and delete their own posts, and track item status.

Administrator Users: Can monitor posts, manage users, resolve disputed claims, and view analytics from the same app via role-based routing.

CHAPTER 2: PROBLEM STATEMENT AND ANALYSIS

2.1 Problem Statement

University campuses lack a dedicated and structured system for managing lost and found items. Students currently depend on informal channels such as Telegram groups and word-of-mouth communication to report and search for lost belongings. This approach is disorganized, unreliable, and ineffective, resulting in many items never being returned to their rightful owners.

2.2 Survey Analysis

A survey was conducted among 12 university students to understand the current state of lost and found management on campus. The following findings were recorded:

Most Commonly Lost Items:

Item	Count	Percentage
Keys	6	50%
ID Card	4	50%
Charger	2	25%
Bag	1	12.5%

Earphone	1	12.5%

Most Common Locations of Loss:

Location	Count	Percentage
Classroom	3	37.5%
Library	2	25%
Unknown	2	25%
Cafeteria	1	12.5%
Lab	1	12.5%

How Students Try to Find Lost Items:

Method	Count	Percentage
Posted on Telegram	5	62.5%
Asked Friends	3	37.5%
Gave Up	0	0%

Behavior When Finding Someone Else's Item:

Action	Count	Percentage
Posted on Telegram	6	75%
Left it there	1	12.5%
Kept it	1	12.5%

Willingness to Use a Dedicated App:

Response	Count	Percentage
Definitely Yes	4	50%
Maybe	2	25%
No	2	25%

Most Wanted Features:

Feature	Count	Percentage
Post photo of item	7	87.5%
Get notified when similar item posted	6	75%
Claim button to request item	4	50%
Search by category/location	4	50%
Message the person who found it	3	37.5%

2.3 Key Findings

- 100% of surveyed students had lost something on campus.
- Keys and ID cards are the most frequently lost items.
- Classrooms and libraries are the most common locations for losing items.
- Telegram is currently the primary method used to report lost and found items, showing a clear need for a dedicated platform.
- 75% of finders post on Telegram but have no structured way to connect with the owner.
- The hardest part of recovery is not knowing who to ask or where to report.
- 87.5% of students want the ability to post photos of lost/found items.

2.4 As-Is Workflow: How Students Currently Recover Lost Items

Before FindIt, students followed a fragmented, informal process to recover lost items. The following narrative describes the typical workflow and identifies where it breaks down at each stage.

Step 1: Student loses item — The student notices an item is missing, often only after leaving the location where it was lost. There is no system prompt, no record, and no timestamp for when or where the item was last seen.

Step 2: Checks pockets and retraces steps — The student physically returns to locations they visited. This works only if the item is still there; if another person has already picked it up, the student has no way to know. Breakdown: No visibility into whether someone found the item.

Step 3: Asks nearby people — The student verbally asks classmates and friends if they have seen the item. This is limited to the student's immediate social circle and the people physically present at the time. Breakdown: Reach is too narrow; a stranger who found the item is never contacted.

Step 4: Posts on Telegram group — The student types a text message describing the item in a campus Telegram group. The post has no photo requirement, no standard format, no location tag, and no category. It is mixed in with unrelated messages and quickly scrolls out of view. Breakdown: Unstructured posts are easily missed; no matching or notification system exists.

Step 5: Waits and follows up manually — The student must actively re-check the Telegram group, re-post to keep the query visible, and manually ask if anyone saw their message. There is no automated alert if a matching item is posted later. Breakdown: No notifications, no persistence, no tracking; the burden stays entirely on the student.

Step 6: Gives up — After a short period of manual effort with no result, most students abandon the search. Items found by strangers remain unclaimed because neither party has a reliable channel to connect. Breakdown: Total process failure — item is permanently lost despite being physically recoverable.

FindIt directly addresses each of these failure points: structured posts with photos and location tagging replace informal Telegram messages; smart match notifications eliminate the need for manual follow-up; and the claim and verification system provides a secure, trusted channel for finder and owner to connect.

CHAPTER 3: OBJECTIVES AND GOALS

3.1 Primary Objective

The primary objective of this project is to design and develop a mobile application called FindIt that provides university students with a centralized, structured, and efficient platform for reporting lost items, posting found items, and facilitating the return of lost belongings through a secure verification and communication system.

3.2 Specific System Goals

Goal 1: Centralized Item Reporting

To provide students with a simple interface to post lost or found items with photos, descriptions, categories, and location information. Users can also edit or delete their own posts at any time.

Goal 2: Efficient Item Discovery

To enable students to search and filter posted items by category, location, and status to quickly find relevant lost or found reports. Urgency badges highlight critical items like ID cards and exam permits.

Goal 3: Secure Claim and Verification System

To implement a multi-layered ownership verification system. When a finder posts a found item, they set category-specific secret questions. A claimant must correctly answer these questions before the finder confirms or declines. Upon confirmation, the owner's phone number is revealed to arrange a meetup and item status is updated to Returned.

Goal 4: Dispute Resolution

To handle cases where multiple users claim the same item. Only one claim is active at a time. For unresolved disputes, either party can escalate to the administrator who makes the final decision.

Goal 5: Smart Match Notification

To implement a notification system that automatically alerts users when a new item post matches the description, category, or location of their lost item report.

Goal 6: Direct Anonymous Communication

To provide integrated in-app messaging that allows finder and owner to communicate without exposing personal contact information until a claim is confirmed.

Goal 7: Status Tracking

To implement a clear visual status tracking system (Lost to Reported to Match Found to Claimed to Returned) so users can monitor the progress of their reports.

Goal 8: Administrative Oversight and Analytics

To provide administrators with tools to manage user accounts, moderate posts, resolve disputes, ban users, and view analytics including total recovered items and recovery success rates.

Goal 9: Auto-Expiry

To automatically archive unresolved posts after 30 days to keep the database clean and relevant.

CHAPTER 4: SCOPE OF THE PROJECT

4.1 System Scope

The scope of the FindIt project covers the development of a mobile application that facilitates the reporting, discovery, and recovery of lost items within a university campus. The system supports the complete process from posting a lost or found item, through secure ownership verification, to final status update upon item return.

4.2 Functional Scope

User registration, authentication, and profile management using JWT authentication.

Role-based access control: student and admin users share the same app but are routed to different screens based on their role stored in MongoDB.

Lost and found item posting with photos, descriptions, categories, and location tagging.

Post editing and deletion by the original poster.

Category-specific secret question system set by the finder at time of posting.

Claim system: claimant answers secret questions, finder confirms or declines, phone number revealed on confirmation.

Dispute escalation to admin when multiple claims cannot be resolved between users.

Search and filtering of items by category, location, and status.

Smart notification system matching lost and found posts by keywords, category, and location.

Urgency badge highlighting for critical items (ID card, exam permit, wallet).

In-app anonymous messaging between users.

Visual status timeline: Lost to Reported to Match Found to Claimed to Returned.

Admin dashboard with user management, post moderation, dispute resolution, and analytics.

4.3 Project Boundaries and Limitations

The following constraints define the boundaries of the current version of FindIt. Each limitation reflects a deliberate design decision, a real-world technical constraint, or a scoping choice made to keep the project achievable within the available timeframe. Understanding these boundaries is important for evaluating the system honestly and planning future iterations.

1. Android-Only Deployment

FindIt is developed and tested on Android (API level 21 and above). Although Flutter is a cross-platform framework and iOS deployment is theoretically possible from the same codebase, iOS builds require an Apple Developer account and macOS toolchain, neither of which was available during this project. In the context of the target campus, Android is by far the dominant platform among students, making this a practical rather than a significant functional constraint. iOS support is identified as a priority for a future release.

2. Internet Connectivity Required

All core functions — posting items, searching, claiming, messaging, and receiving notifications — require an active internet connection. This is because the data layer relies entirely on Firebase Firestore, Firebase Storage, and Firebase Cloud Messaging, which are cloud-hosted services. In a campus setting, reliable Wi-Fi is generally available in classrooms, libraries, and common areas, making this a minor constraint in practice. However, students in areas with poor connectivity may experience delays. Offline mode, which would cache recent posts for browsing without a connection, is noted as a future enhancement rather than a current requirement.

3. No Financial Transactions

The system does not process or facilitate any financial transactions. While item owners may optionally mention a reward in the text description of their post, no payment gateway, wallet system, or in-app currency has been implemented. This decision was made deliberately: integrating a payment system introduces significant regulatory, security, and trust complications that are outside the scope of a campus lost-and-found tool. The primary motivation for returning found items on a university campus is goodwill rather than financial incentive, and adding monetary transactions would risk incentivising dishonest behaviour (e.g., finders withholding items to extract payment). Keeping rewards text-only and informal preserves the social trust that the system depends on.

4. In-Person Verification for Keys and Physically Non-Unique Items

Certain item categories — most notably keys — cannot be verified digitally because they carry no unique identifier such as a serial number or IMEI. For these items, the secret question mechanism used for phones and laptops is not

applicable. Instead, FindIt uses a hybrid approach: the app arranges an in-person meetup through the anonymous chat system, and ownership is confirmed face-to-face by visual inspection of the item. This is a real-world constraint of the problem domain rather than a system deficiency. No purely digital system can fully eliminate the need for physical handover of a physical object; FindIt acknowledges this honestly and provides the infrastructure to make that handover safe and structured.

5. Photos Only; No Video Uploads

Item posts support photo attachments only. Video uploads are excluded for two practical reasons: storage cost and upload time. A single video file may be 10–50 times larger than a photo, which would rapidly exceed Firebase Storage free-tier limits at campus scale and create unacceptably slow upload experiences on mobile data connections. For the purpose of identifying a lost item, a clear photograph is sufficient and is already the most commonly requested feature in the survey (87.5% of respondents). Video would add storage complexity without a proportionate gain in identification accuracy.

6. No Integration with Campus Security or Administration Systems

FindIt operates as a standalone application and does not connect to any existing campus security systems, student registration databases, or ID card systems. Integration with institutional systems would require formal API access agreements, data sharing policies, and IT department involvement that are not feasible within a student project scope. As a result, user identity is verified only through email registration; there is no automatic validation that a registered user is a current enrolled student. In a real deployment, this could be addressed by restricting registration to university email domains (e.g., @university.edu.et), which would be a low-cost mitigation. Full institutional integration remains a long-term goal.

7. No Automated Image-Based Item Matching

The smart match notification system currently matches posts using text-based criteria: category, keywords in the title and description, and location. It does not perform visual comparison of uploaded photos. AI-based image recognition that could automatically match a photo of a found bag to a photo of a lost bag would require a trained machine learning model, a model hosting infrastructure, and significantly more development time than this phase allows. Text-based matching covers the majority of useful matching scenarios given that categories and descriptions are mandatory fields. Image recognition is listed as a future enhancement and would substantially increase match accuracy in a subsequent version.

4.3 Project Boundaries and Limitations

The following constraints define the boundaries of the current version of FindIt.

1. Android-Only Deployment

FindIt is developed and tested on Android (API level 21 and above). Although React Native is a cross-platform framework and iOS deployment is theoretically possible from the same codebase, iOS builds require an Apple Developer account and macOS toolchain, neither of which was available during this project. iOS support is identified as a priority for a future release.

2. Internet Connectivity Required

All core functions require an active internet connection. This is because the data layer relies entirely on cloud-hosted services. In a campus setting, reliable Wi-Fi is generally available in classrooms, libraries, and common areas. Offline mode is noted as a future enhancement.

3. No Financial Transactions

The system does not process or facilitate any financial transactions. No payment gateway, wallet system, or in-app currency has been implemented. Keeping rewards text-only and informal preserves the social trust that the system depends on.

4. In-Person Verification for Keys and Physically Non-Unique Items

Certain item categories — most notably keys — cannot be verified digitally because they carry no unique identifier. For these items, FindIt uses a hybrid approach: the app arranges an in-person meetup through the anonymous chat system, and ownership is confirmed face-to-face.

5. Photos Only; No Video Uploads

Item posts support photo attachments only. Video uploads are excluded for practical reasons: storage cost and upload time. For the purpose of identifying a lost item, a clear photograph is sufficient.

6. No Integration with Campus Security or Administration Systems

FindIt operates as a standalone application and does not connect to any existing campus security systems or student registration databases. User identity is verified only through email registration.

7. No Automated Image-Based Item Matching

The smart match notification system currently matches posts using text-based criteria: category, keywords in the title and description, and location. Image recognition is listed as a future enhancement.

4.4 Future Scope

Future enhancements may include AI-based automatic item matching using image recognition, integration with campus security systems, a reputation and trust rating system for users who regularly return found items, and offline mode support.

CHAPTER 5: SYSTEM BENEFITS AND FEASIBILITY

5.1 System Benefits

Centralized Platform: All lost and found reports are stored in one place, replacing disorganized Telegram posts.

Faster Recovery: Smart match notifications alert users immediately when a matching item is posted.

Secure Verification: Category-specific secret questions and phone number revealed only upon confirmed claim prevent fake claiming.

Dispute Resolution: Admin escalation ensures fair resolution when multiple users claim the same item.

Anonymous Communication: In-app messaging protects user privacy until ownership is verified.

Photo Evidence: Item photos increase chances of correct identification.

Precise Location Tagging: Location fields help pinpoint where items were lost or found.

Accountability: Status timeline and claim system create a structured and trackable recovery process.

Clean Database: Auto-expiry of posts after 30 days keeps the system relevant and uncluttered.

Admin Oversight: Analytics and moderation tools keep the platform trustworthy and safe.

5.2 Feasibility Study

The system is technically feasible. React Native is a well-established cross-platform mobile framework. Node.js/Express provides a reliable backend,

and MongoDB Atlas offers a scalable cloud database. All technologies are industry-standard and free to use at the required scale

5.2.2 Economic Feasibility

The project is economically feasible. React Native, Node.js, and MongoDB Atlas free tier are all sufficient for development and campus-level deployment. No software licensing costs are required.

5.2.3 Operational Feasibility

The system is designed to be intuitive and easy to use. Students can register, post items, search, and claim without specialized training. The admin panel provides straightforward tools for moderation and oversight.

CHAPTER 6: REQUIREMENTS ENGINEERING

6.1 Functional Requirements

6.1.1 User Authentication and Account Management

- FR-01: Users can register using their name, email address, phone number, and password.
- FR-02: Users can log in securely using their registered email and password via Firebase Authentication.
- FR-03: The system routes users to the home screen or admin dashboard based on their role stored in Firestore.
- FR-04: Users can log out securely.
- FR-05: Users can view and update their profile information.

6.1.2 Lost and Found Item Management

- FR-06: Users can post a lost item report with a title, description, category, optional photo, date, and location (campus area or building name).
- FR-07: Users can post a found item report with a title, description, category, photo, date, location (campus area or building name), and category-specific secret questions.
- FR-08: Users can edit or delete their own posts at any time.
- FR-09: Each post displays a status badge: Lost, Found, Claimed, or Returned.
- FR-10: Critical item categories (ID card, exam permit, wallet) display an urgency badge.

- FR-11: Posts automatically archive after 30 days if unresolved.

FR-11b: All newly submitted posts (lost and found) are placed in a pending state and require admin approval before being visible to other users on the platform. The admin receives a notification for each pending post and can approve or reject it. The poster is notified of the admin's decision.

6.1.3 Search and Discovery

- FR-12: Users can search all posts by item name or description keywords.
- FR-13: Users can filter posts by category (keys, ID, phone, bag, charger, laptop, other).
- FR-14: Users can filter posts by location or campus area.
- FR-15: Users can filter posts by status (Lost, Found, Claimed, Returned).

6.1.4 Claim and Verification System

- FR-16: Users can press a This is Mine button on a found item post to initiate a claim.
- FR-17: The system presents the claimant with the secret questions set by the finder.
- FR-18: The finder receives the claimant's answers and can confirm or decline the claim.
- FR-19: Upon confirmation, the owner's phone number is revealed to the finder to arrange item return.

6.1.6 Notification and Matching System

- FR-20: Item status is updated to Returned after confirmation.
- FR-21: If a claim is declined, the post remains active for other claimants.
- FR-22: Secret questions are category-specific. Laptops require serial number, phones require IMEI, bags require contents, keys display an in-person verification message.

6.1.5 Dispute Resolution

- FR-23: Only one claim can be active at a time per post.
- FR-24: Either party can escalate a dispute to the admin using a Report Dispute button.

- FR-25: Admin can review dispute details, read both parties' messages, and make a final decision.
- FR-26: The system sends a push notification when a new post matches the category and location of a lost item report.
- FR-27: The system sends a notification when a claim request is received by the finder.
- FR-28: The system sends a notification when a claim is confirmed or declined.

6.1.7 Messaging System

- FR-29: Users can send and receive in-app messages with other users.
- FR-30: Phone numbers remain hidden in chat until a claim is confirmed.
- FR-31: Unread messages are clearly indicated with a notification badge.

6.1.8 Administrative Module

- FR-32: Admins can view all posts (including pending, active, and archived), review pending posts submitted for approval, and view all registered users.
- FR-33: Admins can approve or reject pending posts. Admins can also delete inappropriate active posts and suspend or ban users. Posters are notified when their post is approved or rejected.
- FR-34: Admins can review and resolve disputed claims by reading claim answers and making a final decision.
- FR-35: Admins can view analytics: total posts, recovered items, recovery rate, most lost categories.

6.2 Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	App loads posts within 3 seconds under normal network conditions.
NFR-02	Usability	Interface is simple and usable without training by any student.
NFR-03	Security	User data and phone numbers are stored securely. Phone numbers revealed only upon confirmed claims.

NFR-04	Reliability	System maintains 99% uptime using Firebase cloud infrastructure.
NFR-05	Scalability	System supports growing numbers of users and posts without performance degradation.
NFR-06	Compatibility	App runs on Android devices with API level 21 or higher.
NFR-07	Privacy	Phone numbers are never publicly displayed; only revealed after claim confirmation.

CHAPTER 7: SYSTEM ANALYSIS AND MODELING

7.1 Use Case Descriptions

This use case describes how a student who has found a lost item reports it on the FindIt platform. The process is designed to ensure that only the true owner can claim the item by requiring the finder to set secret verification questions at the time of posting, preventing fraudulent claims.

Use Case 1: Post Found Item with Secret Questions

Field	Description
Use Case Name	Post Found Item
Actor	Student (Finder)
Precondition	User is logged in
Main Flow	User taps Post, selects Found, fills in title, description, selects category, uploads photo, enters location (campus area or building name), adds category-specific secret questions, and submits. Post is sent to admin for approval before being published.
Postcondition	Found item post submitted and sent to admin for approval. Once the admin approves, the post is published on the platform and the system checks for matching lost posts and sends notifications.
Alternative Flow	If category is Keys, secret question section is replaced with an in-person verification message.

This use case covers the ownership verification process from the perspective of the student who lost an item. The system is designed so that personal contact information is never exposed until a claim is successfully verified, protecting both parties' privacy throughout the transaction.

Use Case 2: Claim an Item

Field	Description
Use Case Name	Claim an Item
Actor	Student (Owner)
Precondition	User is logged in and a found item post exists with no active claim.

Main Flow	Owner taps This is Mine, system displays secret questions, owner submits answers, finder reviews answers and confirms or declines.
Postcondition	If confirmed: phone number revealed, status set to Returned. If declined: post remains active.
Alternative Flow	If multiple users attempt to claim, only one is active at a time. Others must wait for decline.

This use case handles edge cases where a claim cannot be resolved between the finder and claimant. The dispute escalation mechanism exists to ensure fairness and provide a trusted authority — the administrator — to make a binding final decision when direct resolution fails.

Use Case 3: Dispute Escalation

Field	Description
Use Case Name	Escalate Dispute to Admin
Actor	Student, Admin
Precondition	A claim dispute exists between finder and claimant.
Main Flow	Either party taps Report Dispute. Admin receives alert, reviews claim answers and post details, and makes a final decision.
Postcondition	Admin confirms rightful owner or marks claim as rejected. Status updated accordingly.
Alternative Flow	Admin may request additional proof via in-app message before deciding.

This use case describes the tools available to the system administrator for maintaining platform integrity. Admin moderation powers are intentionally broad, covering post removal, user suspension, banning, and analytics review, to keep FindIt a trustworthy and safe environment for all students.

Use Case 4: Admin Moderation

Field	Description
Use Case Name	Admin Moderation
Actor	Admin
Precondition	Admin is logged in and routed to admin dashboard.

Main Flow	Admin views list of all posts and users, selects a post or user to review, and takes action: delete post, suspend user, or ban user.
Postcondition	Post removed or user account suspended or banned. Affected user notified.
Alternative Flow	Admin views analytics dashboard showing recovery rates and most lost categories.

Figure 1: Use Case Diagram - FindIt Application

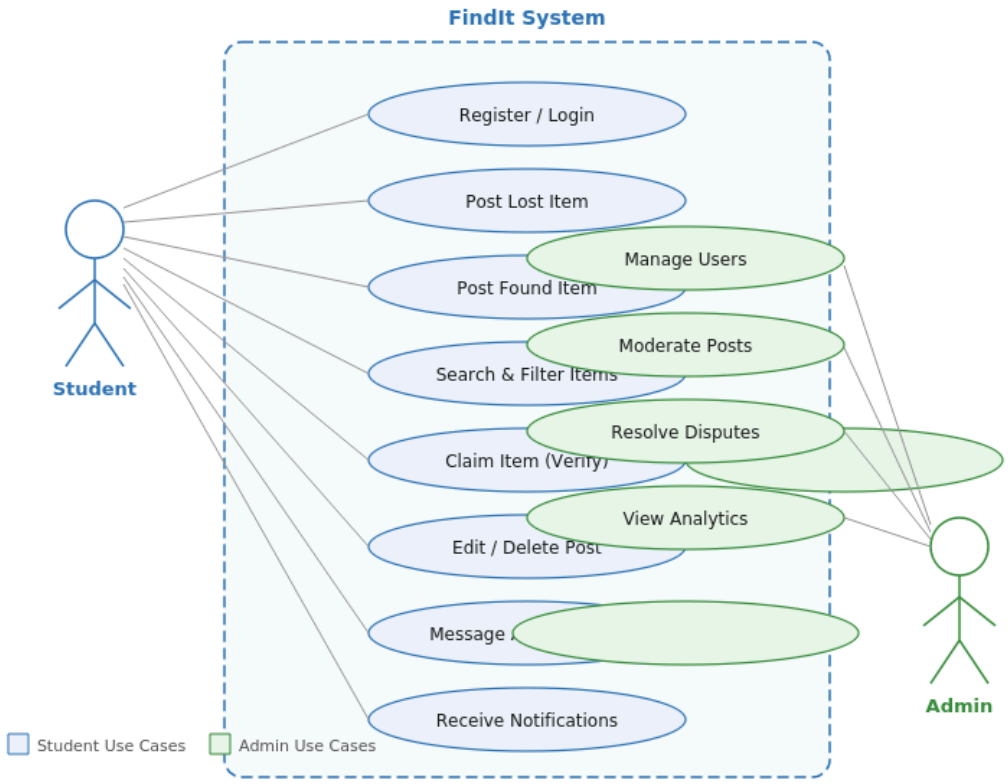

Figure 1: Use Case Diagram – FindIt Campus Lost & Found Application

7.2 Edge Case Analysis

The following table identifies anticipated edge cases in the FindIt system and describes how each is handled. Thorough edge case analysis ensures the system behaves predictably under unusual or adversarial conditions.

#	Edge Case	How the System Handles It
1	Multiple users claim the same found item simultaneously	Only one claim can be active per post at a time (FR-23). Subsequent claimants must wait until the active claim is confirmed or declined before submitting their own.
2	A claimant answers secret questions incorrectly on purpose to block others	The finder reviews submitted answers and can decline an incorrect claim, immediately reopening the post for the next claimant. The dispute escalation path is also available if bad-faith behaviour is suspected.
3	A finder refuses to respond to a valid claim (ignores the confirm/decline prompt)	The claimant can escalate to the admin via the Report Dispute button (FR-24). The admin reviews the submitted answers and post details and makes a binding decision (FR-25).
4	A user posts a found item they did not actually find (fraudulent post)	A fraudulent poster cannot set meaningful secret questions for an item they do not have. Any legitimate owner will fail to have their claim confirmed. The admin can delete the post and suspend the user (FR-33). Photo evidence further deters fabrication.
5	A post expires after 30 days but the item was actually returned informally (outside the app)	Auto-expiry archives the post, keeping the database clean. The poster can manually update the post status to “Returned” before expiry if the item was recovered outside the app.
6	The student who lost an item does not know the exact location where they lost it	Location tagging is optional for lost item posts. The user can set a general area or leave location blank. Smart match notifications rely primarily on category keywords, so a missing location does not prevent matching.
7	A user loses a key with no unique identifying details (no serial number or IMEI)	Keys are handled as a special category with in-person verification. The secret question section is replaced with a message instructing the parties to arrange a physical meetup via the in-app chat. The finder can confirm identity visually.
8	A user account is banned while they have an active claim or open dispute	Banning is performed by the admin, who has full visibility of active claims and disputes before taking action. The admin can resolve any open disputes simultaneously or transfer the decision before applying the ban (FR-33, FR-34).

#	Edge Case	How the System Handles It
9	Two users post the same lost item (duplicate reports)	Each post is independent. The admin can delete a duplicate post during routine moderation. Only one claim is ever active per post, so duplicate posts do not create conflicting claim states.
10	A student loses internet connectivity mid-claim or mid-chat	Firebase Firestore uses local persistence, so partially composed data is preserved. Claims that were not submitted remain as drafts locally. The claim status is only committed once the submission reaches the server, preventing partial writes that could corrupt claim state.

7.3 Database Schema

Users Collection

Field	Type	Description
userId	String	Unique user ID (Firebase Auth UID)
name	String	Full name of the user
email	String	Email address
phoneNumber	String	Phone number (hidden until claim confirmed)
profilePhoto	String	URL to profile photo
createdAt	Timestamp	Account creation date
role	String	student or admin
isBanned	Boolean	Whether user account is banned

Items Collection

Field	Type	Description
itemId	String	Unique item post ID
userId	String	ID of the user who posted
type	String	lost or found

title	String	Item name or title
description	String	Detailed description of the item
category	String	keys / ID / phone / bag / charger / laptop / other
photoUrl	String	URL to item photo (optional for lost, required for found)
location	String	Campus area or building name where item was lost or found
locationName	String	Human-readable location name
status	String	pending_approval / rejected / lost / found / claimed / returned / expired
urgency	Boolean	True for ID card, exam permit, wallet
secretQuestions	Array	Category-specific secret questions set by finder (found items only)
createdAt	Timestamp	Date and time of posting
expiresAt	Timestamp	Auto-expiry date 30 days after posting

Claims Collection

Field	Type	Description
claimId	String	Unique claim ID
itemId	String	ID of the claimed item
claimantId	String	ID of the user claiming the item
answers	Array	Claimant answers to secret questions
status	String	pending / confirmed / declined / disputed
createdAt	Timestamp	Time claim was submitted

Messages Collection

Field	Type	Description
chatId	String	Unique chat ID combining two user IDs

senderId	String	ID of the message sender
receiverId	String	ID of the message receiver
message	String	Message content
timestamp	Timestamp	Time the message was sent
isRead	Boolean	Whether the message has been read

Notifications Collection

Field	Type	Description
notificationId	String	Unique notification ID
userId	String	ID of user to notify
itemId	String	ID of related item post
type	String	match / claim_received / claim_confirmed / claim_declined / dispute / post_approved / post_rejected
message	String	Notification message text
isRead	Boolean	Whether notification has been read
createdAt	Timestamp	Time notification was created

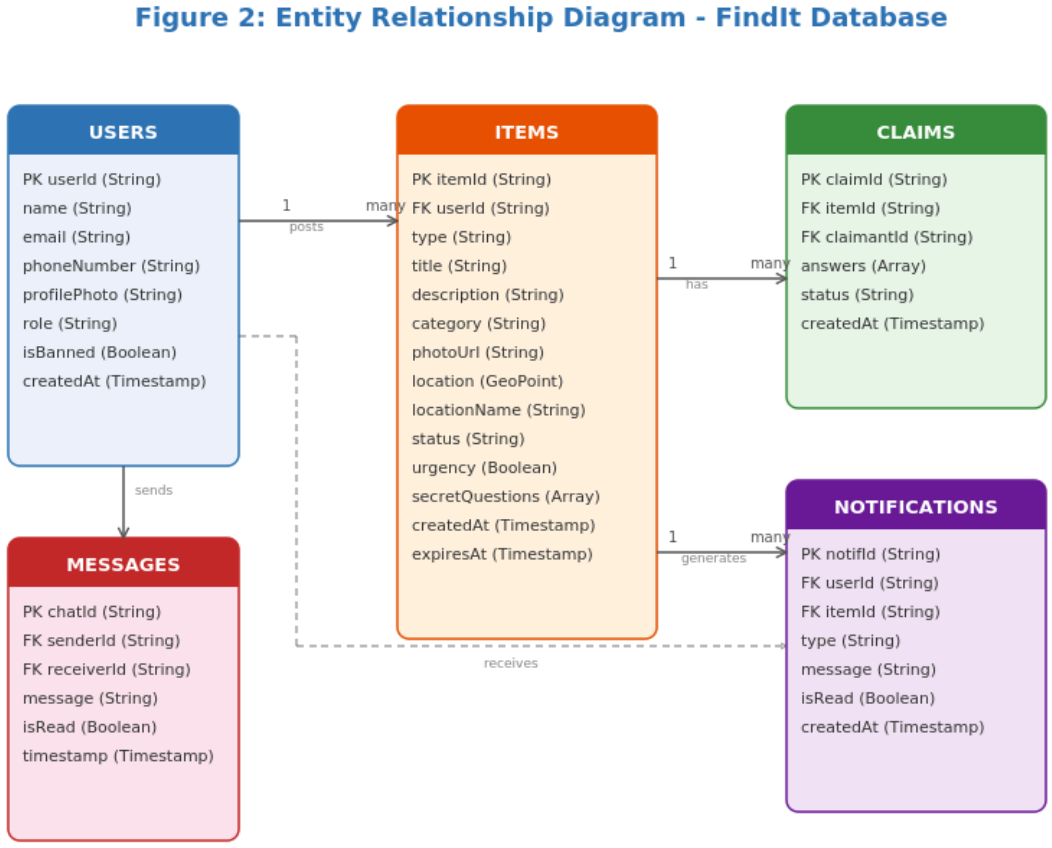

Figure 2: Entity Relationship Diagram (ERD) – FindIt Database Schema

CHAPTER 8: SYSTEM ARCHITECTURE AND DESIGN

8.1 System Architecture

The FindIt application follows a three-tier mobile architecture:

Presentation Layer: React Native mobile application providing the user interface and handling all user interactions including posting, searching, claiming, and messaging.

Business Logic Layer: Node.js/Express services and controllers handling claim verification logic, smart match notification triggering, role-based routing, and auto-expiry scheduling.

Data Layer: MongoDB (via Mongoose) for the cloud database, Node.js/Express + Multer for image storage, JWT/bcrypt for user authentication, and an in-app notification system.

8.2 Technology Stack

Component	Technology	Purpose
Mobile Frontend	React Native (JavaScript)	Cross-platform mobile UI
Authentication	JWT Authentication	Secure login, registration, and role-based routing
Database	MongoDB	Cloud database for posts, claims, and messages

File Storage	Node.js/Express + Multer	Store and retrieve item photos
In-App Notifications	In-app notification system	Match, claim,anddispute notifications
State Management	React Context API	Efficient app state management
Backend	Node.js + Express	REST API and business logic
Real-Time	Socket.io	Real-time chat messaging
Hosting	Render.com+MongoDB Atlas	Cloud hosting and database

8.3 Application Screens

Screen	Description
Splash Screen	App logo and loading screen on startup
Login Screen	Email and password login. Routes to Home or Admin Dashboard based on role.
Register Screen	New user registration form including phone number

Home Screen	Feed of all lost and found posts with urgency badges and status indicators
Post Detail Screen	Full post details including claim button, status timeline, and location information
Post Item Screen	Form to post lost or found item with secret question setup for found items
Edit Post Screen	Allow poster to edit post details or delete the post
Search and Filter Screen	Advanced search by keywords, category, location, and status
Claim Screen	Secret question answering form for claimants
Claim Review Screen	Finder reviews claimant answers and confirms or declines
Chat Screen	In-app anonymous messaging between users
Notifications Screen	Match alerts, claim updates, and dispute notifications
Profile Screen	User profile showing their posted items and activity
Admin Dashboard	Analytics: total posts, recovered items, recovery rate, most lost categories
Admin User Management	List of all users with suspend and ban controls
Admin Post Moderation	List of all posts with delete controls
Admin Post Approval	List of all pending posts awaiting approval. Admin can view post details, approve (publish to platform) or reject (notify poster with reason). Approved posts become visible to all users.
Admin Dispute Resolution	Disputed claims with finder and claimant answers and decision controls

8.4 Claim Verification Flow

- Step 1: Finder posts a found item, sets category-specific secret questions, and submits for admin review.
- Step 2: Admin receives a notification of the pending post, reviews it for accuracy and compliance, and approves or rejects it.
- Step 3: Upon admin approval, the post is published on the platform and the finder is notified. The system checks for matching lost item reports and sends notifications to relevant users.

- Step 4: Owner sees the approved post and taps This is Mine to initiate a claim. Owner answers the secret questions without seeing the expected answers.
- Step 5: Finder receives a notification and reviews the submitted answers.
- Step 6a (Confirmed): Owner's phone number is revealed to the finder. Status changes to Returned.
- Step 6b (Declined): Post remains active. Next claimant can attempt to claim.

Step 7 (Disputed): If unresolved, either party can escalate to the admin who makes the final decision.

8.5 Category-Specific Secret Questions

Category	Verification Method
Laptop	Serial number of the device
Phone	IMEI number of the device
ID Card	Student ID number and department
Wallet	What cards or items are inside
Bag	What specific items are inside the bag
Keys	In-person physical verification: app arranges meetup via chat
Other	Custom questions set by the finder based on item details

CHAPTER 9: CONCLUSION

The FindIt Campus Lost and Found application addresses a real and widespread problem faced by university students. Through a structured survey of 12 students, it was established that 100% had experienced losing items on campus and that current informal recovery methods through Telegram groups are inefficient and unreliable.

The proposed solution, FindIt, provides a dedicated mobile platform built with React Native, Node.js, and MongoDB that enables students to post lost and found items with photos and location information, search and filter posts,

receive smart match notifications, verify ownership through category-specific secret questions, communicate anonymously through in-app messaging, and track item status from report through to confirmed return. All posts are subject to admin approval before being published, ensuring the platform remains safe, accurate, and trustworthy.

A key innovation of the system is the secure claim and verification flow. Finders set secret questions when posting found items, claimants must answer correctly before a claim can be confirmed, and only after confirmation is the owner's phone number revealed. For keys, which have no hidden identifying details, the app arranges an in-person meetup through the chat system. For disputed claims, admin escalation provides a fair and structured resolution mechanism.

The system is technically, economically, and operationally feasible. It leverages industry-standard technologies that are free to use, well-documented, and scalable. The application is designed to be intuitive and accessible to all students regardless of technical background.